VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.

LAB REPORT
on

Data Structures using C Lab

(23CS3PCDST)

Submitted by

Shaikh Uzair Ahmed (1BM23CS307)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Data Structures using C Lab (23CS3PCDST)" carried out by **Shaikh Uzair Ahmed (1BM23CS307),** who is bonafide student of **B.M.S.College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academicrequirements in respect of Data Structures using C Lab (23CS3PCDST) work prescribed for the said degree.

Lab faculty Incharge Namratha M Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Github Link:
https://github.com/Shaikh-Uzair-Ahmed/1BM23CS307

Program 1

Write a program to simulate the working of stack using an array with the following:

a) Push

b) Pop

c) Display

The program should print appropriate messages for stack overflow, stack underflow

Stacks (Pseudo Code)

```
- Initialize an array -> arr
- make following functions.
- size = -1      - max = User defined / your choice
~~push~~
isEmpty()
- if size == -1
    return True
  else
    return False

isFull()
if size == full
  return True
else
  return False

Push(item)
  if isFull()
    return "Array Full"
  else
    size += 1
    arr[size] = item

pop()
  if isEmpty()
    return "Array Empty"
  else
    return arr[size-1]
    size -= 1

Top()
return arr[max-1]

display()
for (i = (Max-1), i>=0,
     i--)
print (arr[i])
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

int top = -1;


int isEmpty(int arr[]) {
    return top == -1;
}

int isFull(int arr[], int limit) {
    return top == limit - 1;
}

int Top(int arr[]) {
    if (isEmpty(arr)) {
        printf("Stack is empty.\n");
        return -1;
    }
    return arr[top];
}

void Display(int arr[]) {
    if (isEmpty(arr)) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack elements: ");
    for (int i = top; i >= 0; i--) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}


void Push(int value, int arr[], int limit) {
    if (isFull(arr, limit)) {
        printf("Stack is full.\n");
    } else {
        top++;
        arr[top] = value;
        printf("Pushed %d onto the stack.\n", value);
```

```
    }
}

void Pop(int arr[]) {
    if (isEmpty(arr)) {
        printf("Stack is empty.\n");
    } else {
        printf("Popped %d from the stack.\n", arr[top]);
        top--;
    }
}

int main() {
    int limit;

    printf("Enter the limit of the stack: ");
    scanf("%d", &limit);

    int arr[limit];

    while (1) {
        int choice, value;
        printf("\nStack Operations:\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Top\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                Push(value, arr, limit);
                break;
            case 2:
                Pop(arr);
                break;
            case 3:
                printf("Top element is: %d\n", Top(arr));
                break;
            case 4:
                Display(arr);
                break;
            case 5:
```

```
                exit(0);
            default:
                printf("Invalid choice. Please try again.\n");
        }
    }

    return 0;
}
```

```
Enter the limit of the stack: 2

Stack Operations:
1. Push
2. Pop
3. Top
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 5
Pushed 5 onto the stack.

Stack Operations:
1. Push
2. Pop
3. Top
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 4
Pushed 4 onto the stack.

Stack Operations:
1. Push
2. Pop
3. Top
4. Display
5. Exit
Enter your choice: 4
Stack elements: 4 5
```

Output 1

```
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 3
Stack is full.

Stack Operations:
1. Push
2. Pop
3. Top
4. Display
5. Exit
Enter your choice: 2
Popped 4 from the stack.

Stack Operations:
1. Push
2. Pop
3. Top
4. Display
5. Exit
Enter your choice: 4
Stack elements: 5

Stack Operations:
1. Push
2. Pop
3. Top
4. Display
5. Exit
Enter your choice: 5
PS C:\Users\uzair\OneDrive\Desktop\1BM23CS307\DSA>
```

Output 2

Program 2

Write a program to convert a given valid parenthesized in-fix arithmetic expression to post-fix expression.The expression consists of single character operands and the binary operators +,-,*,/.

WAP to convert a given valid paranthesized infix arithmetic expression to postfix expression. The expression consists of single character operands and the binary operators +, -, *, /

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int prec(char c)
 {
   if (c == '^')
      return 3;
   else if (c == '/' || c == '*')
      return 2;
   else if (c == '+' || c == '-')
      return 1;
   else
      return -1;
 }

char associativity(char c)
 {
    if (c == '^')
       return 'R';
    return 'L';
 }

void infixToPostfix (const char *s) {
   int len = strlen(s)
   char result [len + 1];
   char stack [len];
```

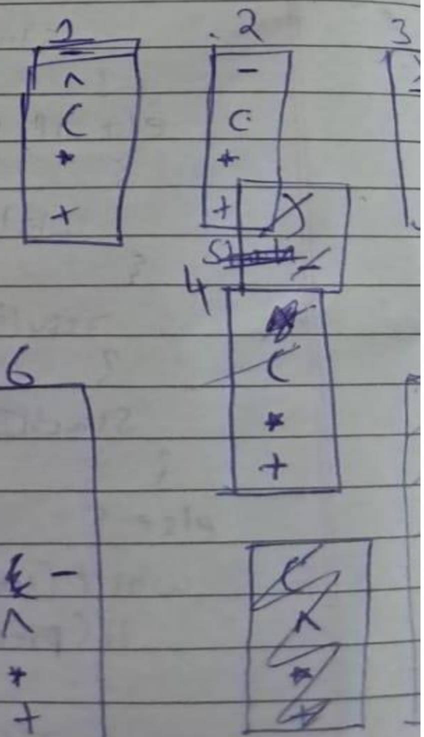
```
int  resultIndex = 0;
int  stackIndex = -1;

if ( !result || !stack){
    printf ("Memory allocation failed! \n");
    return;
}

for (int i = 0; i < len; i++) {
    char c = s[i];

    if((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')
        || (c >= '0' && c <= '9'))
    {
        result[resultIndex++] = c;
    }
    else if (c == '(') {
        stack[++stackIndex] = c;
    }
    else if (c == ')')
    {
        while(stackIndex >= 0 && stack[stackIndex] != '(')
    {
        result[resultIndex++] = stack[stackIndex --];
    }
    stackIndex --;
    }
    else {
    while (stackIndex >= 0 && (prec(c) < prec(stack[stackIndex
        || (prec(c) == prec(stack[stackIndex]) && associativity(c)
        == 'L')) {
          result[resultIndex++] = stack [stackIndex --];
      }
```

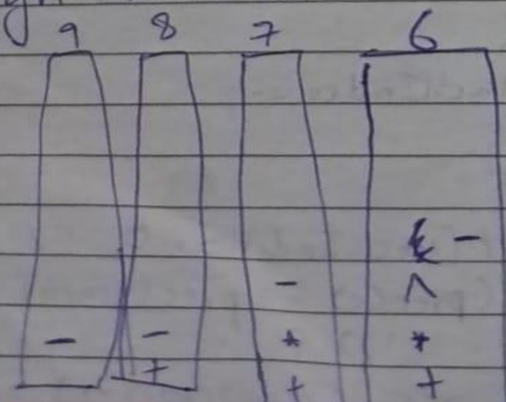
```
stack[++stackIndex] = c;
}//end for//end else
}//end function // end for

while (stackIndex >= 0){
    result[resultIndex++] = stack[stackIndex--];
}

result[resultIndex] = '\0';
printf("%s\n", result);

int main(){
    char exp[] = "a+b*(c^d-e)^(f+g*h)-i";
infixToPostfix(exp);
return 0;
}
```

Output:

abcd^e-fgh*+^*+i-

abcd^e-fgh*+^*+i-





Namdle M.
7/10/2023

Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

int prec(char c){
    if (c == '+' || c == '-') {
        return 1;
    }
    if (c == '*' || c == '/') {
        return 2;
    }
    if (c == '^') {
        return 3;
    }
    return 0;
}

char associativity(char c){
    if(c=='^')
        return 'R';
    return 'L';
}

void infixtopostfix(const char *s){
    int len = strlen(s);
    char result[len+1];
    char stack[len];
    int resultIndex=0;
    int StackIndex=-1;

    if(!result || !stack){
        printf("Memory Allocation Failed \n");
        return;
    }

    for (int i = 0; i < len; i++)
    {
        char c=s[i];
        if ((c>='a' && c<='z') || (c>='A' && c<='Z'))
        {
            result[resultIndex++] = c;
        }
```

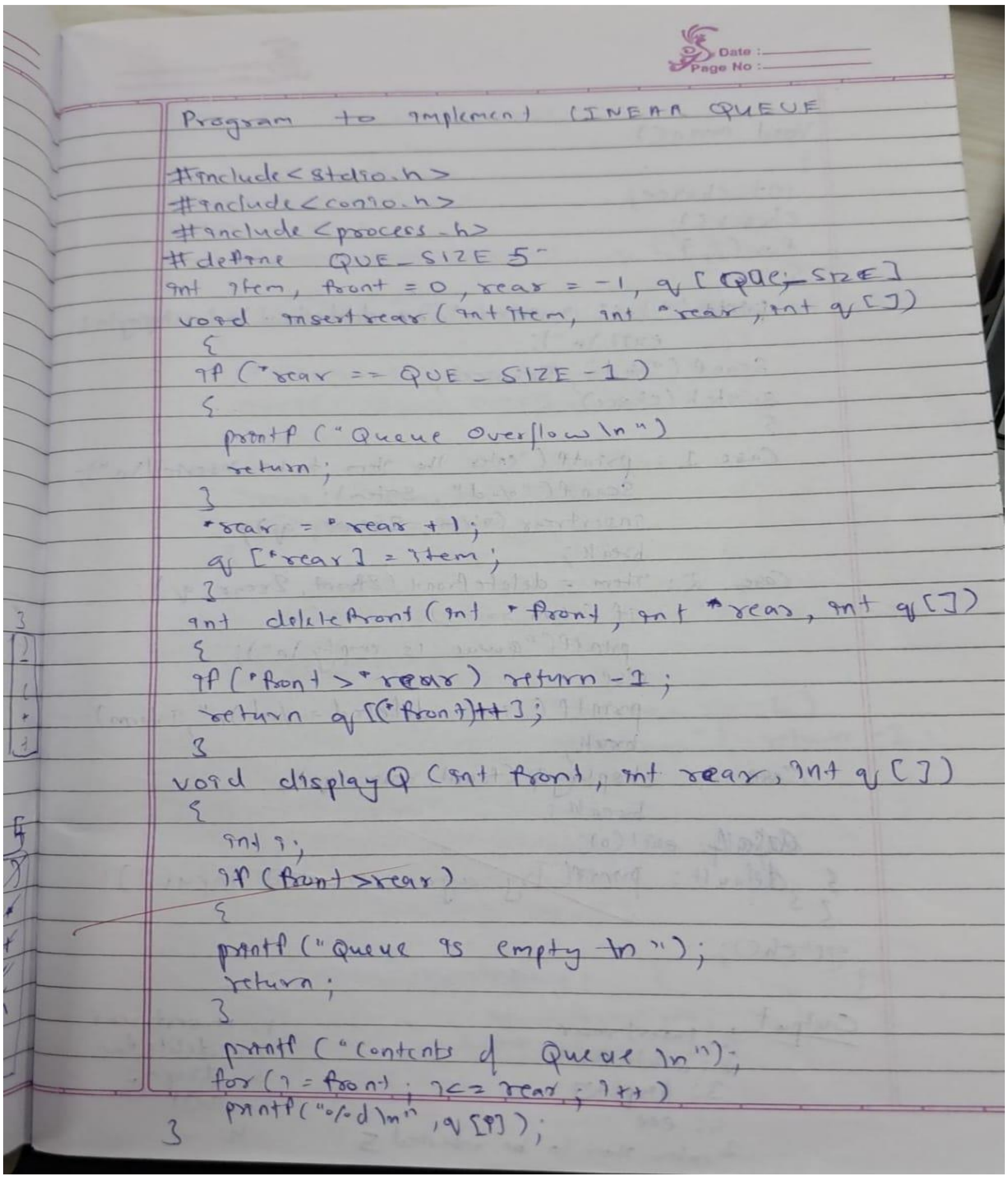
```
        else if(c=='('){
            stack[++StackIndex]=c;
        }
        else if(c==')'){
            while (StackIndex>=0 && stack[StackIndex]!='(')
            {
                result[resultIndex++]=stack[StackIndex--];
            }
           StackIndex--;
        }
        else{
            while (StackIndex>=0 && (prec(c))<prec(stack[StackIndex]) ||
(prec(c)==prec(stack[StackIndex]) && associativity(c)=='L'))
            {
                result[resultIndex++]=stack[StackIndex--];
            }
            stack[++StackIndex]=c;
        }
    }

    while (StackIndex>=0)
    {
        result[resultIndex++]=stack[StackIndex--];
    }
    result[resultIndex++]='\0';
    printf("%s\n",result);
}

int main(){
    char exp[] = "a+b*(c^d-e)^(f+g*h)-i";
    infixtopostfix(exp);
    return 0;
}
```

```
PS C:\Users\uzair\OneDrive\Desktop\1BM23CS307\DSA> cd "c:\Users\uzair\OneDrive\Desktop\1BM23CS307\DSA\" ; if ($?) { gcc infi
xpostfix.c -o infixpostfix } ; if ($?) { .\infixpostfix }
abcd^e-fgh*+^*+i-
```

Output

Program 3

WAP to simulate the working of a queue of integers using an array. Provide the following operations: Insert, Delete, Display
The program should print appropriate messages for queue empty and queue overflow conditions.



```
Date :
Page No :

Program to implement LINEAR QUEUE

#include<stdio.h>
#include<conio.h>
#include<process.h>
#define QUE_SIZE 5
int item, front = 0, rear = -1, q [QUE_SIZE]
void insertrear (int item, int *rear, int q [])
 {
 if (*rear == QUE_SIZE - 1)
 {
  printf ("Queue Overflow \n")
  return;
 }
 *rear = *rear + 1;
 q [*rear] = item;
 }
int deletefront (int *front, int *rear, int q [])
 {
 if (*front > *rear) return -1;
 return q [(*front)++];
 }
void displayQ (int front, int rear, int q [])
 {
  int i;
  if (front >rear)
  {
  printf ("Queue is empty \n");
  return;
  }
  printf ("Contents of Queue \n");
  for (i = front; i<= rear; i++)
  printf("%d\n", q [i]);
 }
```

```
Void main()
{
 int choice;
 clrscr();
 for(;;)
 {
   printf("\n1: insertrear\n2: deletefront\n3: display\n4:
          exit\n");
   scanf("%d", &choice);
   switch(choice)
   {
   case 1: printf("enter the item to be inserted\n");
           scanf("%d", &item);
           insertrear(item, &rear, q);
           break;
   case 2: item = deletefront(&front, &rear, q);
           if (item == -1)
             printf("queue is empty\n");
           else
           printf("item deleted = %d\n", item);
           break;
   case 3: displayQ(front, rear, q);
           break;
   case 4: exit(0);
   default: printf("Try again invalid input");
   }
 }
getch();
}
```

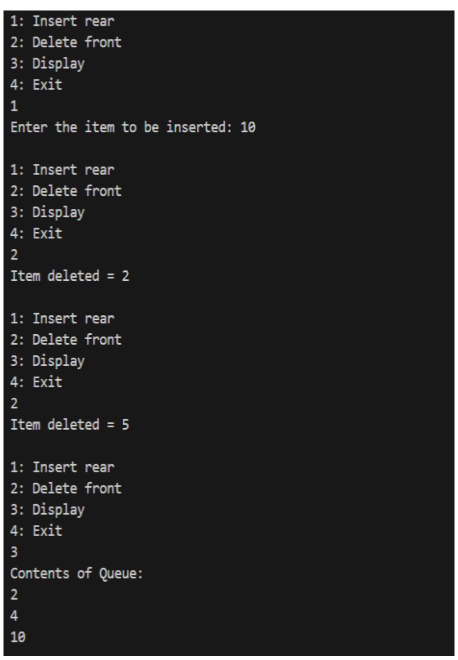
Output

```
1. insertrear
2: delete front
3: display
4: exit
1 enter item to be inserted 5

1. insert rear
2. delete fron
3. display
4: exit
4
```

Code:

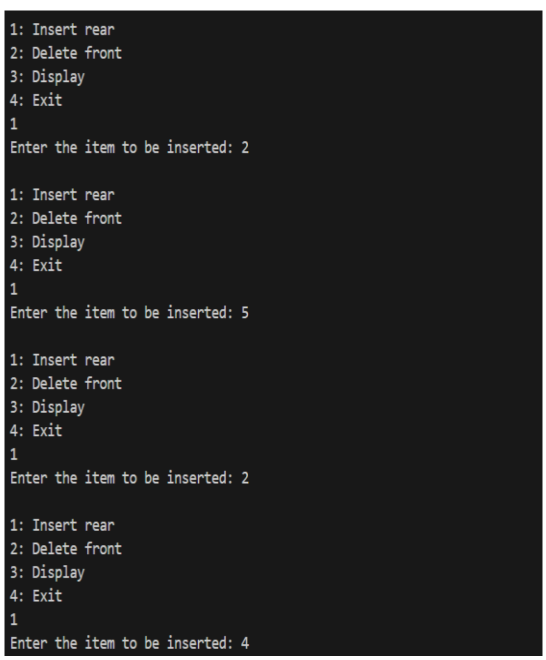
```
#include <stdio.h>
#include <stdlib.h>

#define QUE_SIZE 5

int item, front = 0, rear = -1, q[N];

void insertrear(int item, int *rear, int q[]) {
    if (*rear == N - 1) {
        printf("Queue Overflow \n");
        return;
    }
    *rear = *rear + 1;
    q[*rear] = item;
}

int Deque(int *front, int *rear, int q[]) {
    if (*front > *rear) {
        return -1;
    }
    return q[(*front)++];
}

void displayQ(int front, int rear, int q[]) {
    if (front > rear) {
        printf("Queue is empty \n");
        return;
    }
    printf("Contents of Queue:\n");
    for (int i = front; i <= rear; i++) {
        printf("%d \n", q[i]);
    }
}

int main() {
    int choice;
    for (;;) {
        printf("\n1: Insert rear\n2: Delete front\n3: Display\n4: Exit\n");
        scanf("%d", &choice);
        switch (choice) {
```

```
        case 1:
            printf("Enter the item to be inserted: ");
            scanf("%d", &item);
            insertrear(item, &rear, q);
            break;
        case 2:
            item = Deque(&front, &rear, q);
            if (item == -1)
                printf("Queue is empty\n");
            else
                printf("Item deleted = %d \n", item);
            break;
        case 3:
            displayQ(front, rear, q);
            break;
        case 4:
            exit(0);
        default:
            printf("Invalid choice, please try again.\n");
    }
  }
  return 0;
}
```



```
1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 10

1: Insert rear
2: Delete front
3: Display
4: Exit
2
Item deleted = 2

1: Insert rear
2: Delete front
3: Display
4: Exit
2
Item deleted = 5

1: Insert rear
2: Delete front
3: Display
4: Exit
3
Contents of Queue:
2
4
10
```

Output 1



```
1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 2

1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 5

1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 2

1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 4
```

Output 2

```
1: Insert rear
2: Delete front
3: Display
4: Exit
4
PS C:\Users\uzair\OneDrive\Desktop\1BM23CS307\DSA>
```

Output 3

Program 4

WAP to simulate the working of a circular queue of integers using an array. Provide the following operations: Insert, Delete & Display.
The program should print appropriate messages for queue empty and queue overflow conditions.

LAB 4 Circular Queue

```
#include <stdio.h>
#include <stdlib.h>
#Define N 5
int q[N]

void Enqueue (int item, int *rear, int *front, int q[])
{
    if ((*rear+1)%N == front){
    printf("Queue Overflow")
    return;
    }
    *rear = (*rear+1) % N;
    q[*rear] = item;
}

int deque (int *front, int *rear, int q[])
{
    if (*front == -1 && *rear == -1)
        printf("Queue underflow"); return -1;
        int temp = q[*front];
    else if (*front == *rear) {
        *rear = -1;
        int ;
        *front = -1;
        ; }
    else *front = ((*front+1)%N);
        q[ ];
} return temp;
```

```
void display(~~front~~ int front, int rear, int q[])
{
  int i;
  if (front == -1 && rear == -1)
  {
     ~~return~~ printf("Que is Empty\n");
     return;
  }
  ~~else~~
  {
  printf("Contents of Queue \n");
  for (i = front; i != rear; i = (i+1) % N)
      printf("%d", q[i]);
  printf("%d", q[rear]);
}

void main()
{ int front = -1; int rear = -1;
  int choice;
  ~~clrscr();~~
  for (;;)
  {
  printf("\n 1: Enque ~~insertrear~~ \n 2: deque ~~deletefront~~ \n 3.display \n 4:
        exit \n");
  scanf("%d", &choice);
  switch (choice)
  {
    case 1: printf("Enter item to be inserted\n");
            scanf("%d", &item);
            Enque ~~insertrear~~(item, &rear, q);
            break;
   case 2: item = deque ~~deletefront~~ (&front, &rear, q);
           if (item == -1)
           printf("Queue is empty\n");
```

```
else
printf ("item deleted = %d \n", item);
break;
case 3:
    displayQ (front, rear, q);
    break;
Case 4: exit (0);
default: printf ("Try again wrong input");
}
}
}
```

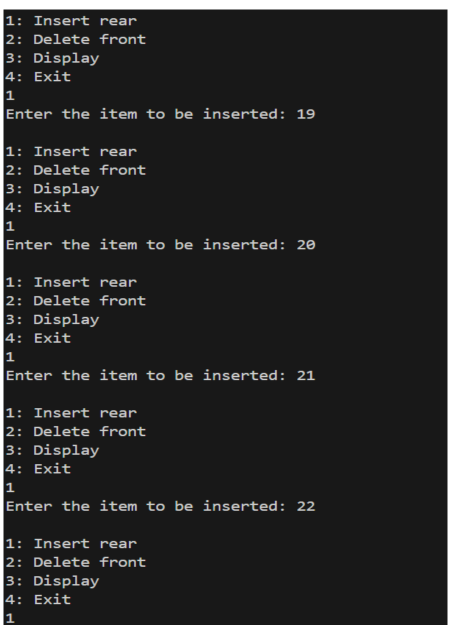
execute Namrata. M 21/10/2024

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define N 5

int q[N];

void Enqueue(int item, int *rear,int *front , int q[]) {
   if (((*rear+1)%N) == *front) {
      printf("Queue Overflow \n");
      return;
   }
   if (*front == -1)
   {
      *front = 0;
   }
```

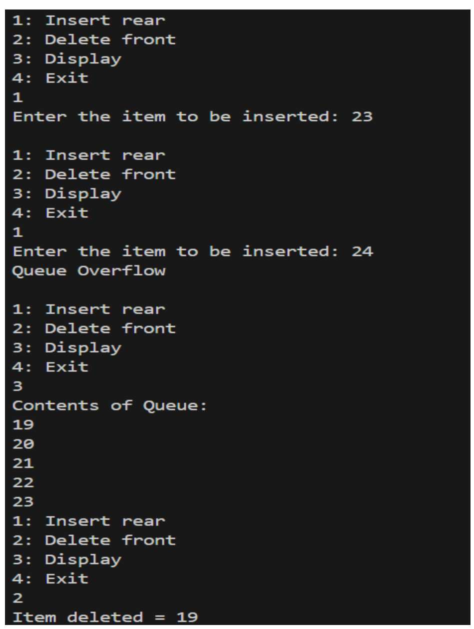
```
    *rear = ((*rear + 1)%N);
    q[*rear] = item;
}

int Deque(int *front, int *rear, int q[]) {
    if (*front == -1 && *rear == -1) {
        return -1;
    }
    int temp = q[*front];
    if(*front == *rear){
        *rear = -1;
        *front = -1;
    }
    else{
        *front = ((*front+1)%N);
    }
    return temp;
}

void displayQ(int front, int rear, int q[]) {
    if (front == -1 &&  rear == -1) {
        printf("Queue is empty \n");
        return;
    }
    printf("Contents of Queue:\n");
    for (int i = front; i != rear; i=(i+1)%N) {
        printf("%d \n", q[i]);
    }
    printf("%d",q[rear]);
}

int main() {
    int choice;
    int rear = -1;
    int front = -1;
    int item;
    for (;;) {
        printf("\n1: Enque\n2: Deque1\n3: Display\n4: Exit\n");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
```

```
            printf("Enter the item to be inserted: ");
            scanf("%d", &item);
            Enqueue(item, &rear, &front , q);
            break;
        case 2:
            item = Deque(&front, &rear, q);
            if (item == -1)
                printf("Queue is empty\n");
            else
                printf("Item deleted = %d \n", item);
            break;
        case 3:
            displayQ(front, rear, q);
            break;
        case 4:
            exit(0);
        default:
            printf("Invalid choice, please try again.\n");
        }
    }
    return 0;
}
```



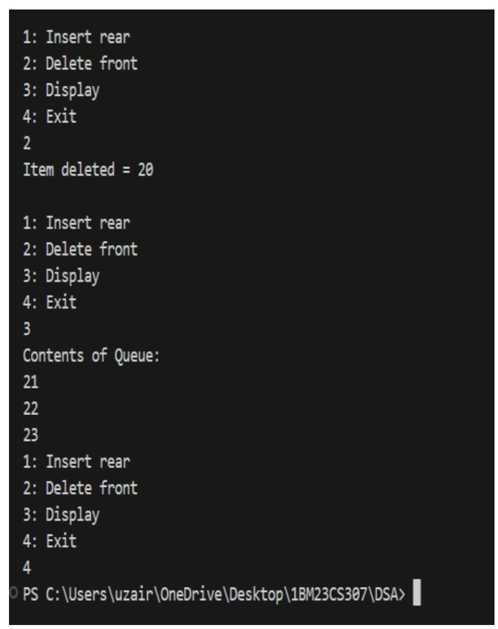
```
1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 19

1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 20

1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 21

1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 22

1: Insert rear
2: Delete front
3: Display
4: Exit
1
```

Output 1



```
1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 23

1: Insert rear
2: Delete front
3: Display
4: Exit
1
Enter the item to be inserted: 24
Queue Overflow

1: Insert rear
2: Delete front
3: Display
4: Exit
3
Contents of Queue:
19
20
21
22
23
1: Insert rear
2: Delete front
3: Display
4: Exit
2
Item deleted = 19
```

Output 2



```
1: Insert rear
2: Delete front
3: Display
4: Exit
2
Item deleted = 20

1: Insert rear
2: Delete front
3: Display
4: Exit
3
Contents of Queue:
21
22
23
1: Insert rear
2: Delete front
3: Display
4: Exit
4
PS C:\Users\uzair\OneDrive\Desktop\1BM23CS307\DSA>
```

Output 3

Program 5

Write a program to Implement Singly Linked List with following operations

a) Create a linked list.

b) Insertion of a node at **first position** and **at end of list.**

Display the contents of the linked list.

Leetcode problem no.20 (Valid parantheses)

54

LAB - 5 (Using Double pointers)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node* link;
};

struct Node* createNode (int val)
{
  struct Node* newNode = (struct Node*) malloc (sizeof (struct Node));
  newNode -> data = val;
  newNode -> link = NULL;
  return newNode;
}

void display (struct Node* head) {
  struct Node* ptr = head;
  while (ptr != NULL) {
     printf ("%d -> ", ptr -> data);
     ptr = ptr -> link;
  }
  printf ("NULL \n");

void insertEnd (struct Node** head, int n) {
  struct Node* temp = createNode (n);
  if (*head == NULL) {
       *head = temp;
  } else {
       struct Node* ptr = *head
       while (ptr -> link != NULL)
          { ptr = ptr -> link; }
  }  } ptr -> link = temp;
```

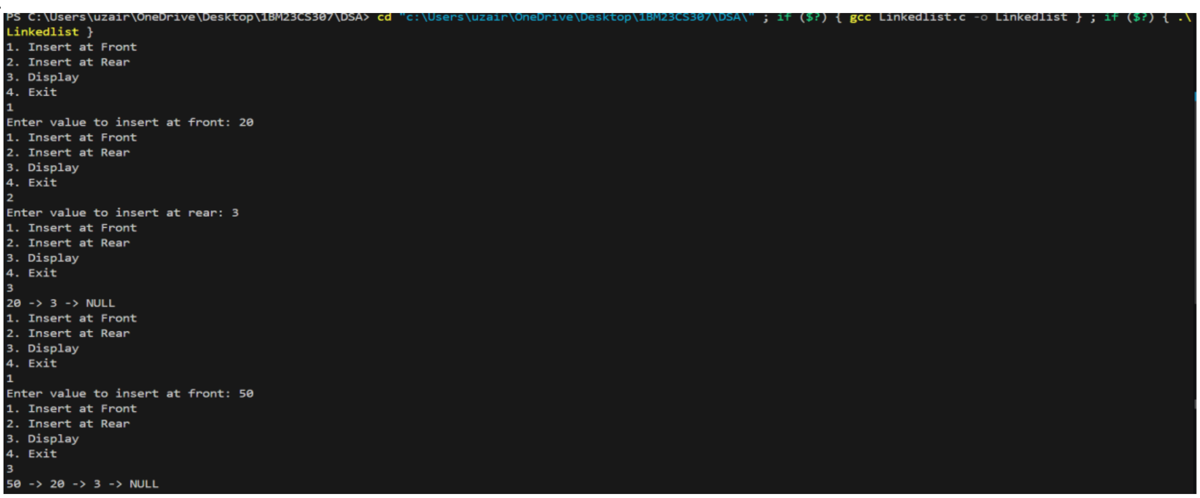
```
void insertFront(struct Node** head, int n)
  {
      struct Node* temp = createNode(n);
      temp -> link = *head;
      *head = temp;
  }

int main ()
{
   struct Node* head = NULL;
   int choice, val;

   while(1) {
    printf("1. Insert Front \n");
    printf("2. Insert at Rear \n");
     "    ("3. Display \n");
     "    ("4. Exit\n");
    scanf("%d", &choice);
   // same indent
   switch (choice) {
      case 1:
            pf ("Enter value to insert at Front: ");
            scanf ("%d", &val);
            insertFront (&head, val);
            break;
      case 2:
            pf ("            insert at Rear : ");
            scanf (          );
            insertEnd (&head, val);
            break.
      case 3:
            display (head);
            break;
```

```
case 4:
   exit(0);
default:
   printf("Invalid choice. please Try again \n");

  }
}
return 0;
}
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* link;
};

struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->link = NULL;
    return newNode;
}

void display(struct Node* head) {
    struct Node* ptr = head;
    while (ptr != NULL) {
        printf("%d -> ", ptr->data);
        ptr = ptr->link;
    }
    printf("NULL\n");
}

void insertEnd(struct Node** head, int n) {
    struct Node* temp = createNode(n);
    if (*head == NULL) {
        *head = temp;  // If the list is empty, the new node becomes the head
    } else {
        struct Node* ptr = *head;
        while (ptr->link != NULL) {
            ptr = ptr->link;
        }
        ptr->link = temp;  // Add the new node at the end
    }
}

void insertFront(struct Node** head, int n) {
    struct Node* temp = createNode(n);
    temp->link = *head;
    *head = temp;  // The new node becomes the head of the list
}

int main() {
```

```
    struct Node* head = NULL;
    int choice, val;

    while (1) {
        printf("1. Insert at Front\n");
        printf("2. Insert at Rear\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert at front: ");
                scanf("%d", &val);
                insertFront(&head, val);
                break;
            case 2:
                printf("Enter value to insert at rear: ");
                scanf("%d", &val);
                insertEnd(&head, val);
                break;
            case 3:
                display(head);
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice. Please try again.\n");
        }
    }
    return 0;
}
```



Output

```c
#include <string.h>

bool isValid(char* s) {
    int n = strlen(s);
    char stack[n];
    int top = -1;
    int i = 0;
    while (i < n)
    {
        char c = s[i];
        if (c == '(' || c == '{' || c == '[')
            stack[++top] = c;
        else
        {
            if (top == -1)
                return false;
            char topChar = stack[top--];
            if ((c == ')' && topChar != '(') ||
                (c == '}' && topChar != '{') ||
                (c == ']' && topChar != '['))
                return false;
        }
        i++;
    }
    return top == -1;
}
```

Leet Code Valid Parenthesis

Program 6

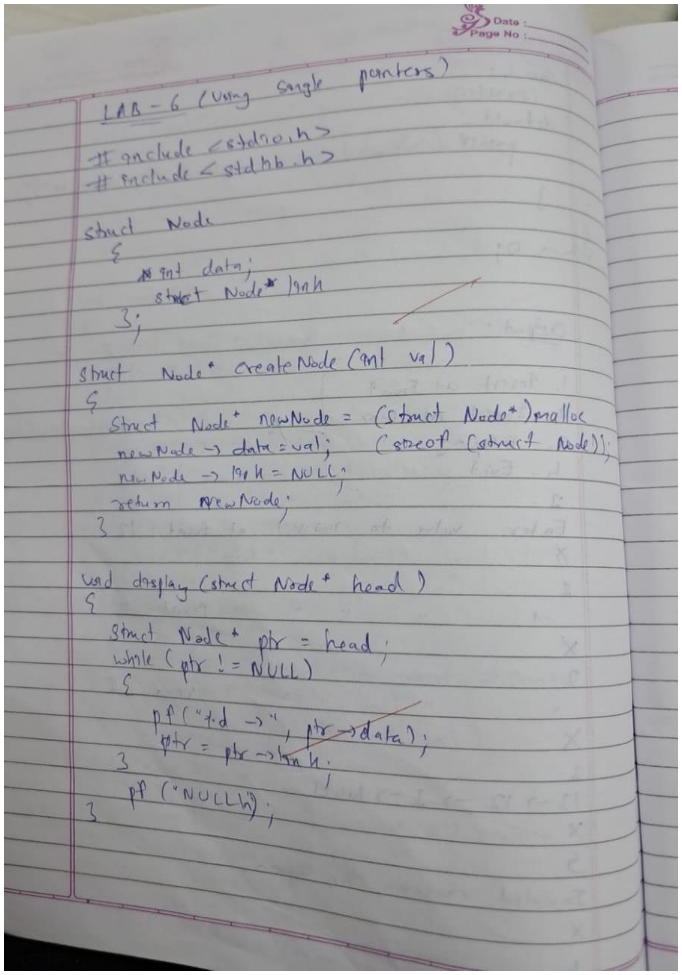
Write a Program to Implement Singly Linked List with following operations

a) Create a linked list.

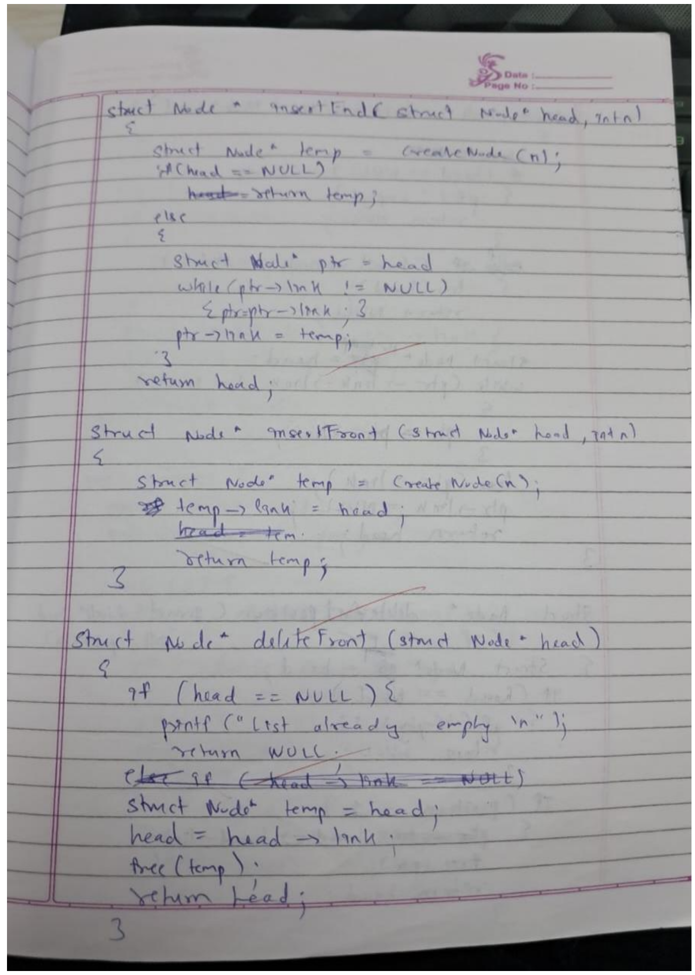
b) Deletion of first element, specified element and last element in the list.

c) Display the contents of the linked list.

Leetcode Problem -- 739 (Daily Temperature)



```
LAB - 6 (Using single pointers)

#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node* link
};

struct Node* createNode (int val)
{
    struct Node* newNode = (struct Node*)malloc
                           (sizeof (struct Node));
    newNode -> data = val;
    newNode -> link = NULL;
    return newNode;
}

void display (struct Node* head)
{
    struct Node* ptr = head;
    while (ptr != NULL)
    {
        pf("%d ->", ptr->data);
        ptr = ptr->link;
    }
    pf ("NULL\n");
}
```



```
struct Node* insertEnd( struct Node* head, int n)
{
    struct Node* temp = createNode (n);
    if(head == NULL)
        return temp;
    else
    {
        struct Node* ptr = head
        while (ptr->link != NULL)
            {ptr=ptr->link;}
        ptr->link = temp;
    }
    return head;
}

struct Node* insertFront (struct Node* head, int n)
{
    struct Node* temp = createNode(n);
    temp -> link = head;
    return temp;
}

struct Node* deleteFront (struct Node* head)
{
    if (head == NULL){
        printf("List already empty \n");
        return NULL.
    struct Node* temp = head;
    head = head -> link;
    free(temp);
    return head;
}
```

```
Struct Node* deleteEnd (struct Node* head)
{
    ~~struct Node~~
    if (head == NULL)
    { ptf ("Empty");
        return NULL;
    }
    if (head -> link == NULL)
    { free (head);
        return NULL;
    }
    struct Node* ptr = head;
    while (ptr -> link -> link != NULL)
    {
        ptr = ptr -> link;
    }
    free (ptr -> link);
    ptr -> link = NULL;
    return head;
}

Struct Node* deleteAtposition (struct Node* head, int position)
{ Struct Node* ptr = head;
  if (head == NULL)
  { pf ("Empty \n");
    return NULL;
  }
  if (position == 1)
  { head = head -> link;
    free (ptr);
    return head;
  }
```

```
for (int i=1; i< position -1 && ptr->link != NULL;
                                                i++)
  {
     ptr = ptr->link;
  }
if (ptr->link == NULL)
{
   pf("Out of bounds. \n");
   return head;
}
struct Node* temp = ptr->link;
ptr->link = temp->link;
free(temp);
return head;
}

int main() {
 struct Node* head = NULL;
 int choice, val, pos;

 while (1) {
  pf("1. Insert at Front \n");
  pf("2.    "    " Rear \n");
  pf("3. Delete Front \n");
  pf("4.  "  Rear \n");
  pf("5. " at Position \n");
  pf("6. Display \n");
  pf("7. Exit \n");
  Scanf("%d", &choice);

  switch (choice)
  {
```

```
case 1:
    printf("Enter value to insert at front: ");
    scanf("%d", &val);
    head = insertFront(head, val);
    break;

Case 2:
    pf("                insert at rear: ");
    scanf( ________ );
    head = insertEnd(head, val);
    break;

Case 3:
    head = deleteFront(head);
    break;

Case 4:
    head = deleteEnd(head);
    break;

Case 5:
    printf("Enter position to delete: ");
    scanf("%d", &pos);
    head = deleteAtPosition(head, pos);
    break;
Case 6:
    display(head);
    break;
Case 7:
    exit(0);
default:
    printf("Invalid choice. Try Again.\n")
switch
  }
while
} return 0;
```

Vaneeta M
11/11/2024

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* link;
};

// Function to create a new node
struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->link = NULL;
    return newNode;
}

// Function to display the linked list
void display(struct Node* head) {
    struct Node* ptr = head;
    while (ptr != NULL) {
        printf("%d -> ", ptr->data);
        ptr = ptr->link;
    }
    printf("NULL\n");
}

// Function to insert at the end of the list
struct Node* insertEnd(struct Node* head, int n) {
    struct Node* temp = createNode(n);

    if (head == NULL) {
        return temp;  // If the list is empty, the new node becomes the head
    } else {
        struct Node* ptr = head;
        while (ptr->link != NULL) {
            ptr = ptr->link;
        }
        ptr->link = temp;  // Add the new node at the end
    }
    return head;
}

// Function to insert at the front of the list
struct Node* insertFront(struct Node* head, int n) {
    struct Node* temp = createNode(n);
    temp->link = head;  // The new node points to the current head
    return temp;        // Return the new node as the new head of the list
}
```

```
// Function to delete the front node
struct Node* deleteFront(struct Node* head) {
    if (head == NULL) {
        printf("The list is already empty.\n");
        return NULL;
    }
    struct Node* temp = head;
    head = head->link;   // Move head to the next node
    free(temp);          // Free the old head
    return head;
}

// Function to delete the end node
struct Node* deleteEnd(struct Node* head) {
    if (head == NULL) {
        printf("The list is already empty.\n");
        return NULL;
    }
    if (head->link == NULL) { // Only one node in the list
        free(head);
        return NULL;
    }
    struct Node* ptr = head;
    while (ptr->link->link != NULL) {
        ptr = ptr->link;
    }
    free(ptr->link);      // Free the last node
    ptr->link = NULL;     // Set the second last node's link to NULL
    return head;
}

// Function to delete a node at a specified position
struct Node* deleteAtPosition(struct Node* head, int position) {
    struct Node* ptr = head;
    if (head == NULL) {
        printf("The list is empty.\n");
        return NULL;
    }
    if (position == 1) { // Delete the first node
        head=head->link;
        free(ptr);
        return head;
    }
    for (int i = 1; i < position - 1 && ptr->link != NULL; i++) {
        ptr = ptr->link;
    }
    if (ptr->link == NULL) {
        printf("Position out of bounds.\n");
```

```
        return head;
    }
    struct Node* temp = ptr->link;
    ptr->link = temp->link;
    free(temp);           // Free the node at the specified position
    return head;
}

int main() {
    struct Node* head = NULL;
    int choice, val, pos;

    while (1) {
        printf("1. Insert at Front\n");
        printf("2. Insert at Rear\n");
        printf("3. Delete Front\n");
        printf("4. Delete Rear\n");
        printf("5. Delete at Position\n");
        printf("6. Display\n");
        printf("7. Exit\n");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert at front: ");
                scanf("%d", &val);
                head = insertFront(head, val);
                break;
            case 2:
                printf("Enter value to insert at rear: ");
                scanf("%d", &val);
                head = insertEnd(head, val);
                break;
            case 3:
                head = deleteFront(head);
                break;
            case 4:
                head = deleteEnd(head);
                break;
            case 5:
                printf("Enter position to delete: ");
                scanf("%d", &pos);
                head = deleteAtPosition(head, pos);
                break;
            case 6:
                display(head);
                break;
            case 7:
                exit(0);
```

```
            default:
                printf("Invalid choice. Please try again.\n");
        }
    }
    return 0;
}
```

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* dailyTemperatures(int* temperatures, int temperaturesSize, int* returnSize) {
    *returnSize = temperaturesSize;
    int* answer = (int*)calloc(temperaturesSize, sizeof(int));
    int* stack = (int*)malloc(temperaturesSize * sizeof(int));
    int top = -1;

    for (int i = 0; i < temperaturesSize; i++) {
        while (top !=-1 && temperatures[i] > temperatures[stack[top]]) {
            int j = stack[top--];
            answer[j] = i - j;
        }
        stack[++top] = i;
    }
    free(stack);
    return answer;

}
```

Leet Code Daily Temperatures

Program 7

a) Write a Program to Implement Single Link List with following operations:

(i)Sort the linked list,

(ii) Reverse the linked list,

(iii)Concatenation of two linked lists.

2/12/2024

Week - 8

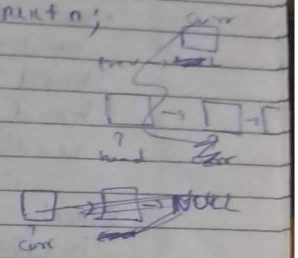
6 a) Single Link list
with
- Sort
- Reverse
- Concatenation operation

b)
- Stack
- Queue

a) Code:

```
#include <stdio.h> #include <stdlib.h>
struct Node
{
    int data;
    struct Node* next;
};

struct Node* CreateNode (int val)
 {
   struct Node* newnode = (struct Node*) malloc (sizeof
                                        (struct Node));
   newnode -> data = val;
   newnode -> next = NULL;
   return newnode;
 }

void sort (struct Node * head)
{
  struct Node *i, *j;
  int temp;
  for (i = head; i -> next != NULL; i = i -> next)
   { for (j = i -> next; j != NULL ; j = j -> next)
      {
        if (i -> data > j -> data)
           { temp = i -> data;
             i -> data = j -> data;
             j -> data = temp
```

```
struct Node *
~~void~~ reverse (struct Node * head)
{
    struct Node *prev, *curr, *nextn;
    prev = NULL;
    curr = head;
    nextn = head;

    while (nextn != NULL)
    {
        nextn = nextn -> next;
        curr -> next = prev;
        prev = curr;
        curr = nextn;
    }
    h~~ead = prev;~~ return prev;
}
```



```
struct Node *
~~void~~ Concat (struct Node *head1, struct Node *head2)
{   if (head1 == NULL) return head2;
    ~~int count = 0;~~
    struct Node *temp1; ~~*temp2~~
    temp1 = head1;
    ~~temp2 = head2;~~
    while (temp1 -> next != NULL)
    {
        temp1 = temp1 -> next
    }
    temp1 -> next = head2;
    return head1;
```

```
void display (struct Node * head)
{   struct Node * ptr = head;
    while (ptr != NULL)
    {   pf ("%d -> " ptr -> data);
        ptr = ptr -> next; }
    pf ("NULL \n");
}
void main ()
{
    struct Node* head = NULL;
    head -> data = 4;
    head -> next = Create Node (2);
    head -> next -> next = Create Node (9);
    head -> next -> next -> next = Create Node (1);

    struct Node* head2 = NULL;
    head2 -> data = 5;
    head2 -> next = Create Node (3);
    head2 -> next -> next = Create Node (8);

    display (head);
    display (head2);

    ~~reverse~~ sort (head);
    display (head);

    head2 = reverse (head2);
    display (head2);

    head = Concat (head1, head2);
    display (head);
}
```

Code:

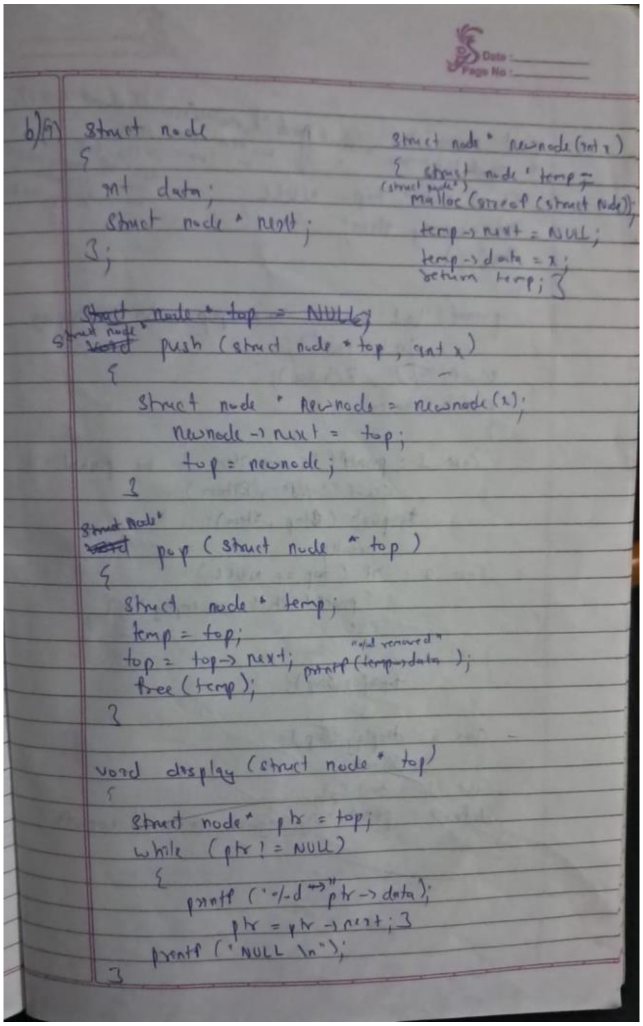
```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* CreateNode(int val) {
    struct Node* newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = NULL;
    return newnode;
}

void sort(struct Node* head) {
    struct Node *i, *j;
    int temp;
    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data) {
                temp = i->data;
                i->data = j->data;
                j->data = temp;
            }
        }
    }
}

struct Node* reverse(struct Node* head) {
    struct Node *prev = NULL, *curr = head, *nextn = NULL;
    while (curr != NULL) {
        nextn = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextn;
    }
    return prev;
}

struct Node* Concat(struct Node* head1, struct Node* head2) {
    if (head1 == NULL) return head2;

    struct Node* temp1 = head1;
    // Traverse to the last node of the first list
    while (temp1->next != NULL) {
        temp1 = temp1->next;
    }
```

```
    temp1->next = head2;
    return head1;
}

void display(struct Node* head) {
    struct Node* ptr = head;
    while (ptr != NULL) {
        printf("%d ->", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = CreateNode(4);
    head->next = CreateNode(2);
    head->next->next = CreateNode(9);
    head->next->next->next = CreateNode(1);

    struct Node* head2 = CreateNode(5);
    head2->next = CreateNode(3);
    head2->next->next = CreateNode(8);
    display(head);
    display(head2);

    sort(head);
    display(head);

    head2 = reverse(head2);
    display(head2);

    head = Concat(head, head2);
    display(head);

    return 0;
}
```

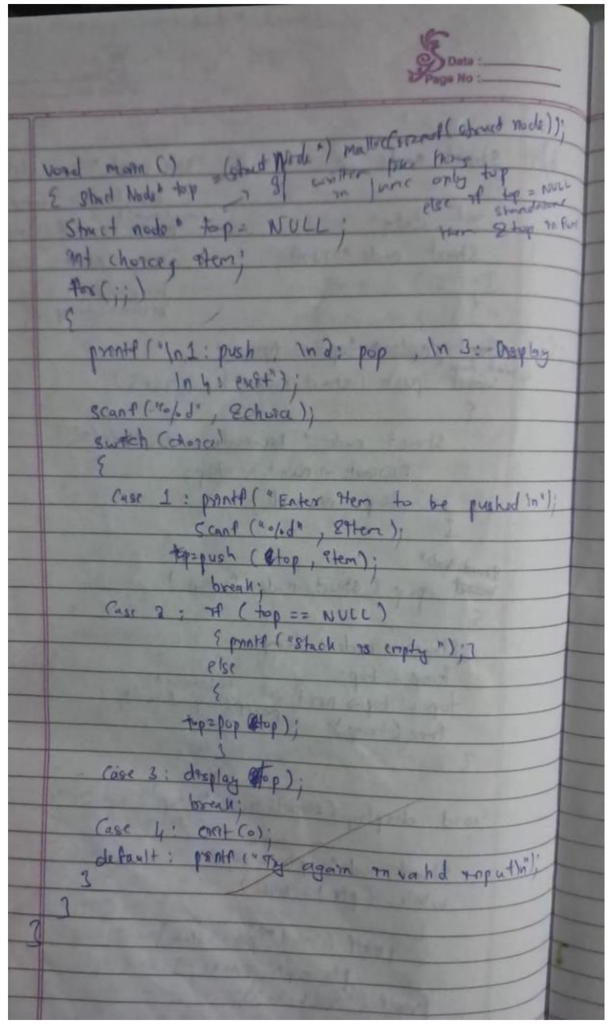
```
PS C:\Users\uzair\OneDrive\Desktop\1BM23CS307> cd "c:\Users\uzair\OneDrive\Desktop\1BM23CS307\DSA\Lab 8\" ;
if ($?) { gcc SLLOperations.c -o SLLOperations } ; if ($?) { .\SLLOperations }
4 ->2 ->9 ->1 ->NULL
5 ->3 ->8 ->NULL
1 ->2 ->4 ->9 ->NULL
8 ->3 ->5 ->NULL
1 ->2 ->4 ->9 ->8 ->3 ->5 ->NULL
PS C:\Users\uzair\OneDrive\Desktop\1BM23CS307\DSA\Lab 8>
```

Output

b)Write a Program to Implement Single Link List to simulate
(i)Stack
(ii)Queue Operations.



```
b)(i) struct node
{
    int data;
    struct node * next;
};

struct node * newnode(int x)
{ struct node * temp = (struct node*)
    malloc(sizeof(struct node));
    temp->next = NULL;
    temp->data = x;
    return temp; }

~~struct node * top = NULL;~~
struct node* ~~void~~ push(struct node * top, int x)
{
    struct node * newnode = newnode(x);
    newnode->next = top;
    top = newnode;
}

struct node*
~~void~~ pop(struct node * top)
{
    struct node * temp;
    temp = top;
    top = top->next;   printf("%d removed", temp->data);
    free(temp);
}

void display(struct node * top)
{
    struct node* ptr = top;
    while (ptr != NULL)
    {
        printf("%d ->", ptr->data);
        ptr = ptr->next; }
    printf("NULL \n");
}
```



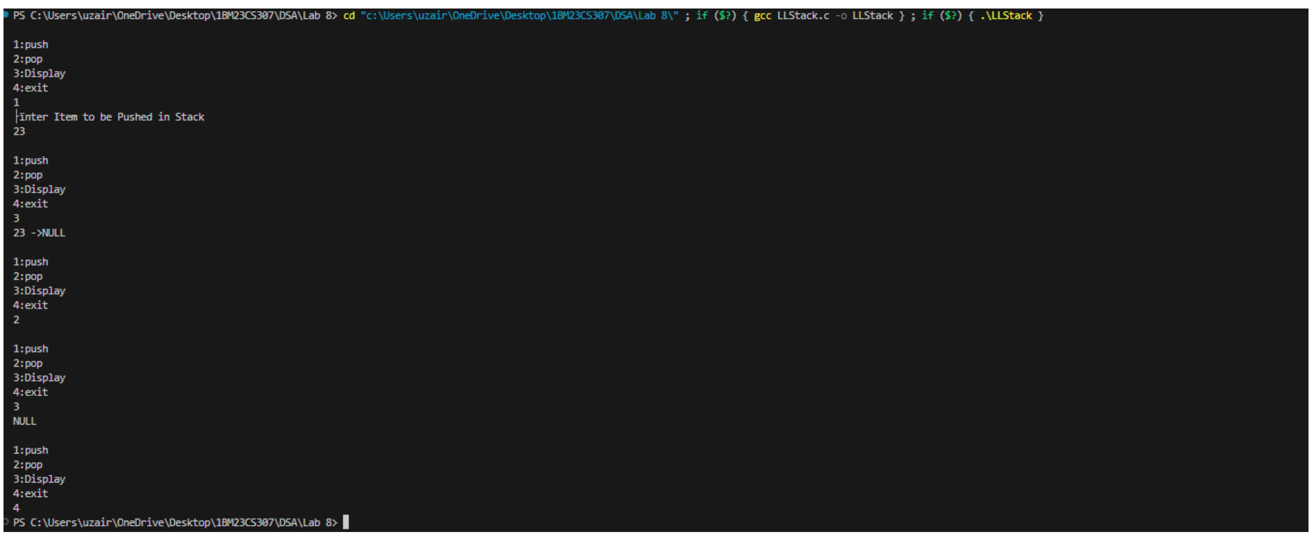
```
void main()
{ struct Node* top = (struct Node*) malloc(sizeof(struct node));
    struct node * top = NULL;
    int choice, item;
    for(;;)
    {
        printf("\n1: push , \n2: pop , \n3: Display
               \n4: exit");
        scanf("%d", &choice);
        switch (choice)
        {
          case 1 : printf("Enter item to be pushed \n");
                   scanf("%d", &item);
                   top=push(&top, item);
                   break;
          case 2 : if (top == NULL)
                   { printf("Stack is empty"); }
                   else
                   {
                   top=pop(&top);
                   }
          case 3 : display(&top);
                   break;
          case 4 : exit(0);
          default : printf("Try again invalid input\n");
        }
    }
}
```

(i) Code:
#include <stdio.h>
#include <stdlib.h>

```
struct Node {
    int data;
    struct Node* next;
};

struct Node* CreateNode(int val) {
    struct Node* newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = NULL;
    return newnode;
```

```
}


struct Node* push(struct Node*top,int x)
{
    struct Node* newnode = CreateNode(x);
    newnode->next = top;
    top = newnode;
    return newnode;
}

struct Node* pop(struct Node* top)
{
    struct Node* temp;
    temp = top;
    top = top->next;
    free(temp);
    return top;

}

void display(struct Node* head) {
    struct Node* ptr = head;
    while (ptr != NULL) {
        printf("%d ->", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

void main()
{
    struct Node* top = (struct Node*)malloc(sizeof(struct Node));
    top = NULL;
    int choice,item;
    for(;;)
    {
        printf("\n1:push \n2:pop \n3:Display \n4:exit\n");
        scanf("%d",&choice);
        switch (choice)
        {
        case 1:
            printf("Ënter Item to be Pushed in Stack \n");
            scanf("%d",&item);
            top = push(top,item);
            break;
        case 2:
            if (top == NULL)
            {
```

```
            printf("Stack is Empty");
          }
          else
          {
            top =pop(top);
          }
          break;
        case 3:
          display(top);
          break;
        case 4:
          exit(0);
        default:
        printf("Try again invalid input \n");
        break;
        }
    }
}
```



Output (i)

```
(ii) struct node
  { same as
    stack
  }

struct node* newnode (int x)
  {
    same as stack
  }

struct node*
~~void~~ deque ( struct node * front, ~~struct node* rear~~)
{
  struct node * temp = front;
  ~~temp = front;~~
  if (front == NULL ~~&& rear == NULL~~)
      printf ("Underflow \n"); return NULL;
  else ~~if (front == rear)~~
    ~~{ temp = front;~~
      ~~front = NULL;~~
      ~~rear = NULL;~~
      ~~printf ("%d removed", temp -> data);~~
      ~~free (temp);~~
    ~~}~~
  else
  {
    front = front -> next;
    free (temp);
  } return front;
}
```

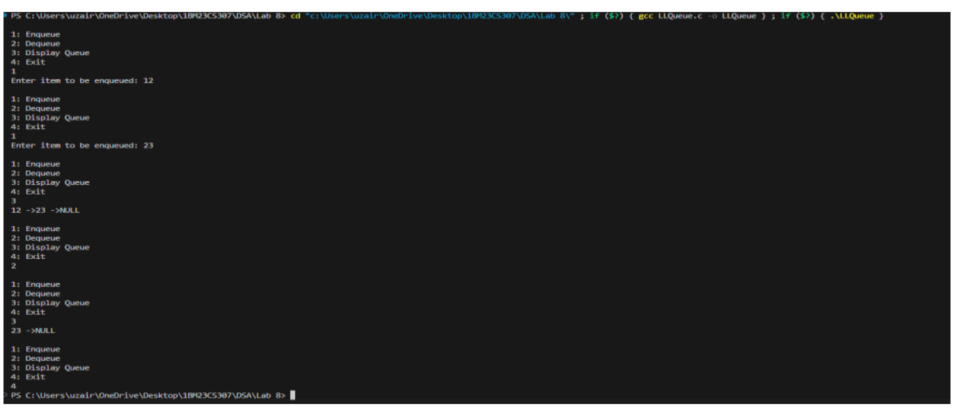
```
struct node*
~~void~~ enque ( ~~struct node* front~~, struct node* rear,
            int x)
{
   struct node* temp = newnode(x);
   if (~~front == NULL~~ && rear == NULL)
   {
      ~~front = temp;~~ return temp;
      ~~rear = temp;~~
      ~~return;~~
   }
   else
   {
      rear -> next = temp;
      rear = temp;
      ~~rear = temp;~~
      return ~~front~~ ~~temp~~ rear;
   }
}

void display(struct node* front)
  {
   same as stack;
  }

void front(struct node* front)
  {
   pf ("%d", front -> data); }
  }

void rear(struct node* rear)
{ printf ("%d", rear -> data); }
```

```
void main ()
{
  struct node* front = NULL;
  struct node* rear = NULL;
  int choice, data;
  for (;;)
  {
    printf ("\n 1: enque , \n 2: deque  \n 3: Display
             \n 4: front  \n 5: Rear  \n 6: exit");
    scanf ("%d", &choice);
    switch (choice)
    {
      case 1: printf ("Enter item to enque ");
              scanf ("%d", &data);
          rear=enque ( ~~front~~, rear, data);  -> if (front == NULL) { front=rear; }
              break;
      case 2 : front = deque (front, ~~rear~~)   if (front == NULL) {
                                                   rear = NULL; }
              break;
      case 3: display (front);
              break;
      case 4: front(front);
              break;
      case 5: rear (rear);
              break;
      case 6 : exit (0);
      default : printf (" Try again invalid input \n");
    }
  }
}
```

(ii) Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* CreateNode(int val) {
    struct Node* newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = NULL;
    return newnode;
}

struct Node* enqueue(struct Node* rear, int x) {
    struct Node* newnode = CreateNode(x);
    if (rear == NULL) {
        return newnode;
    }
    rear->next = newnode;
    return newnode;
}

struct Node* dequeue(struct Node* front) {
    if (front == NULL) {
        printf("Queue is empty\n");
        return NULL;
    }
    struct Node* temp = front;
    front = front->next;
    free(temp);
    return front;
}

void display(struct Node* front) {
    struct Node* ptr = front;
    if (ptr == NULL) {
        printf("Queue is empty\n");
        return;
    }
    while (ptr != NULL) {
        printf("%d ->", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}
```

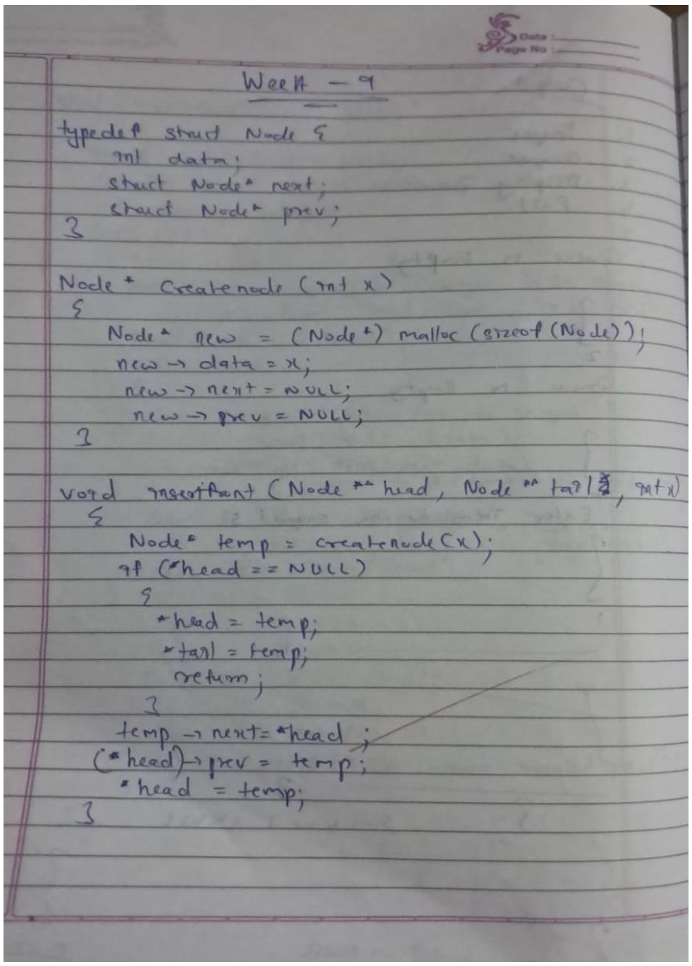
```
int main() {
    struct Node* front = NULL;
    struct Node* rear = NULL;
    int choice, item;

    for (;;) {
        printf("\n1: Enqueue\n2: Dequeue\n3: Display Queue\n4: Exit\n");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            printf("Enter item to be enqueued: ");
            scanf("%d", &item);
            rear = enqueue(rear, item);  // Enqueue operation, rear updated
            if (front == NULL) {
                front = rear;
            }
            break;
        case 2:
            front = dequeue(front);  // Dequeue operation, front updated
            if (front == NULL) {
                rear = NULL;
            }
            break;
        case 3:
            display(front);
            break;
        case 4:
            exit(0);  // Exit the program
        default:
            printf("Invalid input. Please try again.\n");
            break;
        }
    }
}
```



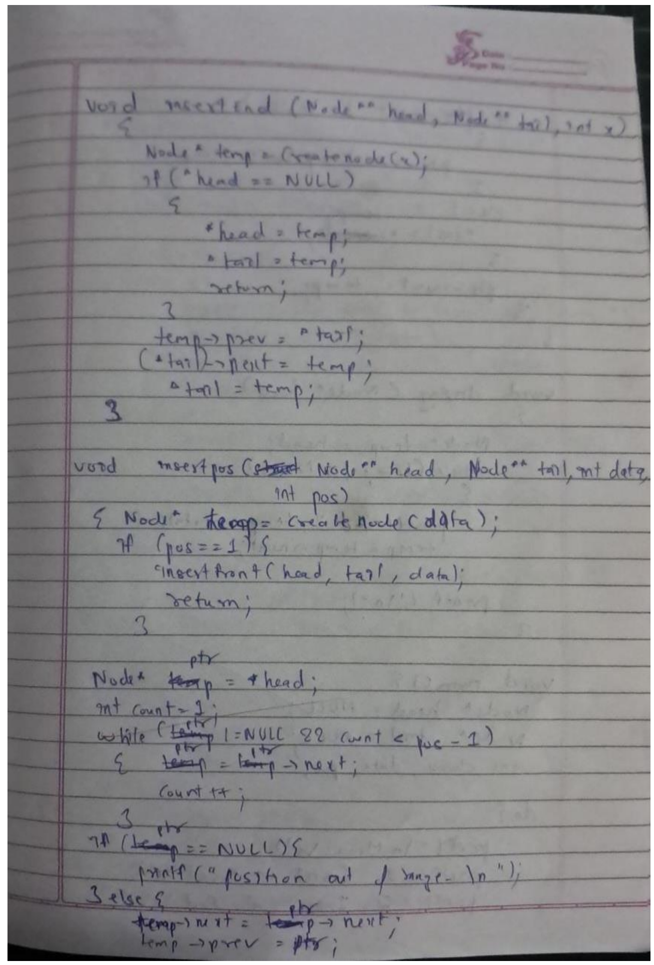
Output (ii)

Program 8

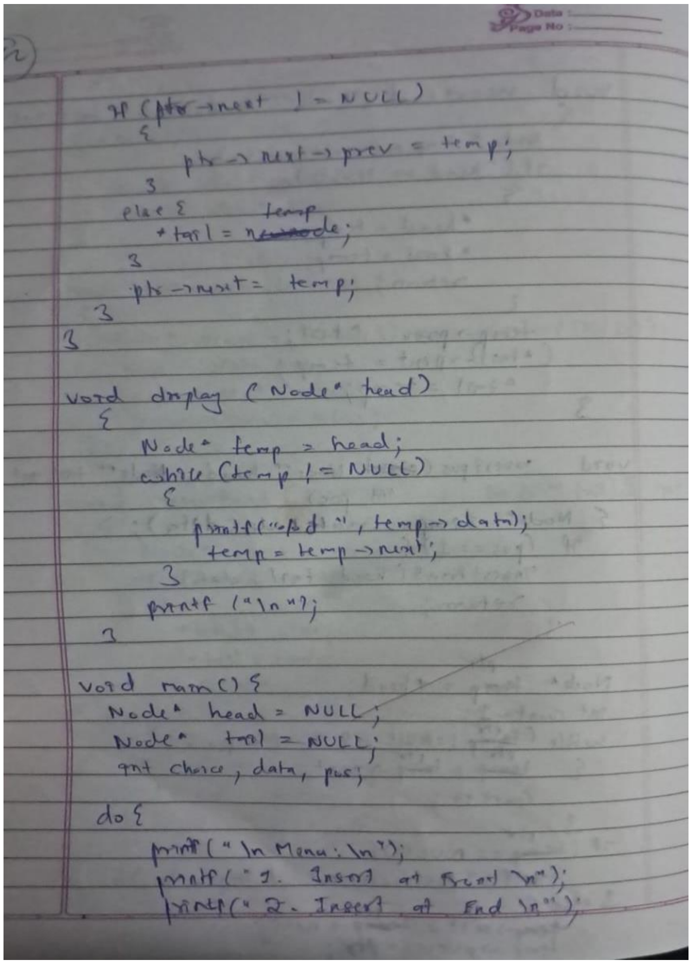
Write a Program to Implement doubly link list with primitive operations
a) Create a doubly linked list.
b) Insert a new node at the beginning.
c) Insert the node based on a specific location
d) Insert a new node at the end.
e) Display the contents of the list



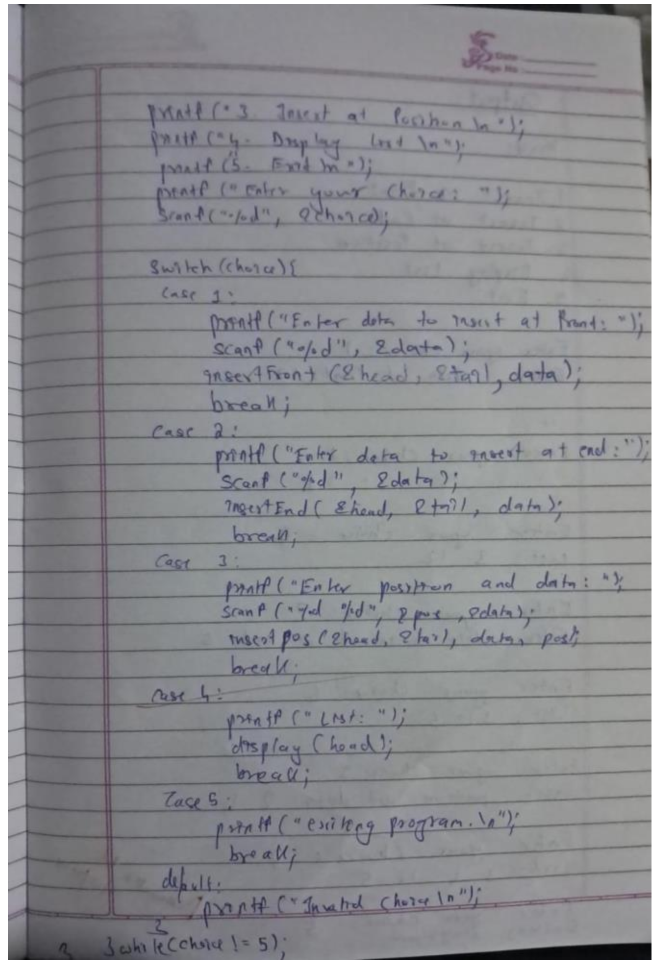
```
Week - 9

typedef struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
}

Node* Createnode (int x)
{
    Node* new = (Node*) malloc (sizeof (Node));
    new -> data = x;
    new -> next = NULL;
    new -> prev = NULL;
}

void insertFront (Node** head, Node** tail, int x)
{
    Node* temp = createnode(x);
    if (*head == NULL)
    {
        *head = temp;
        *tail = temp;
        return;
    }
    temp -> next = *head;
    (*head) -> prev = temp;
    *head = temp;
}
```



```
void insertEnd (Node** head, Node** tail, int x)
{
    Node* temp = Createnode(x);
    if (*head == NULL)
    {
        *head = temp;
        *tail = temp;
        return;
    }
    temp -> prev = *tail;
    (*tail) -> next = temp;
    *tail = temp;
}

void insertpos (struct Node** head, Node** tail, int data,
                int pos)
{ Node* temp = Create Node (data);
  if (pos == 1) {
     insertFront (head, tail, data);
     return;
  }

  Node* ptr = *head;
  int count = 1;
  while (ptr != NULL && count < pos - 1)
  {   ptr = ptr -> next;
      count ++;
  }
  if (ptr == NULL) {
      printf ("position out of range. \n");
  } else {
      temp -> next = ptr -> next;
      temp -> prev = ptr;
```



```
if (ptr->next != NULL)
  {
     ptr->next->prev = temp;
  }
else {
   *tail = ~~newnode~~ temp;
  }
ptr->next = temp;
 }
}

void display (Node* head)
 {
   Node* temp = head;
   while (temp != NULL)
    {
      printf("%d ", temp->data);
      temp = temp->next;
    }
   printf ("\n");
 }

void main() {
  Node* head = NULL;
  Node* tail = NULL;
  int choice, data, pos;

  do {
    printf (" \n Menu:\n");
    printf (" 1. Insert at Front \n");
    printf (" 2. Insert at End \n");
```



```
printf(" 3. Insert at Position \n");
printf(" 4. Display List \n");
printf(" 5. Exit \n");
printf (" Enter your Choice: ");
scanf("%d", &choice);

switch (choice){
 case 1:
   printf("Enter data to insert at front: ");
   scanf ("%d", &data);
   insertFront (&head, &tail, data);
   break;
 case 2:
   printf("Enter data to insert at end: ");
   scanf ("%d", &data);
   insertEnd ( &head, &tail, data);
   break;
 case 3:
   printf("Enter position and data: ");
   scanf ("%d %d", &pos, &data);
   insertPos (&head, &tail, data, pos);
   break;
 case 4:
   printf (" List: ");
   display (head);
   break;
 case 5:
   printf ("exiting program.\n");
   break;
 default:
   printf ("Invalid Choice \n");
 }
} while(choice != 5);
```

Code:

```
#include <stdio.h>
#include <stdlib.h>


typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;


Node* createNode(int data) {
    Node* temp = (Node*)malloc(sizeof(Node));
    temp->data = data;
    temp->prev = NULL;
    temp->next = NULL;
    return temp;
}


void insertFront(Node** head, Node** tail, int data) {
    Node* temp = createNode(data);
    if (*head == NULL) {
```

```
        *head = *tail = temp;
    } else {
        temp->next = *head;
        (*head)->prev = temp;
        *head = temp;
    }
}


void insertEnd(Node** head, Node** tail, int data) {
    Node* temp = createNode(data);
    if (*tail == NULL) {
        *head = *tail = temp;
    } else {
        (*tail)->next = temp;
        temp->prev = *tail;
        *tail = temp;
    }
}


void insertAtPos(Node** head, Node** tail, int data, int pos) {
    Node* temp = createNode(data);


    if (pos == 1) {
        insertFront(head, tail, data);
        return;
    }

    Node* ptr = *head;
    int count = 1;
    while (ptr != NULL && count < pos - 1) {
        ptr = ptr->next;
        count++;
    }

    if (ptr == NULL) {
        printf("Position out of range.\n");
    } else {
        temp->next = ptr->next;
        temp->prev = ptr;

        if (ptr->next != NULL) {
            ptr->next->prev = temp;
        } else {
            *tail = temp;
        }
        ptr->next = temp;
```

```
    }
}


void printList(Node* head) {
    Node* ptr = head;
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}


int main() {
    Node* head = NULL;
    Node* tail = NULL;
    int choice, data, pos;

    do {
        printf("\nMenu:\n");
        printf("1. Insert at Front\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Position\n");
        printf("4. Display List\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter data to insert at front: ");
                scanf("%d", &data);
                insertFront(&head, &tail, data);
                break;
            case 2:
                printf("Enter data to insert at end: ");
                scanf("%d", &data);
                insertEnd(&head, &tail, data);
                break;
            case 3:
                printf("Enter position and data: ");
                scanf("%d %d", &pos, &data);
                insertAtPos(&head, &tail, data, pos);
                break;
            case 4:
                printf("List: ");
                printList(head);
                break;
```

```
            case 5:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 5);

    return 0;
}
```

```
Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 1
Enter data to insert at front: 12

Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 1
Enter data to insert at front: 3

Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 4
List: 3 12
```

Output 1

```
Enter your choice: 4
List: 3 12

Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 2
Enter data to insert at end: 5

Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 4
List: 3 12 5

Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 3
Enter position and data: 2 4
```

Output 2

```
List: 3 12 5

Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 3
Enter position and data: 2 4

Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 4
List: 3 4 12 5

Menu:
1. Insert at Front
2. Insert at End
3. Insert at Position
4. Display List
5. Exit
Enter your choice: 5
Exiting program.
```

Output 3

Program 9

Write a program
8a) To construct a binary Search tree.
8b) To traverse the tree using all the methods i.e., in-order, preorder and post order, display all traversal order

Date :
Page No :

Week - 10

8\. Write a program
a) To construct a binary Search Tree
b) To traverse the tree using all the methods i.e, in-order, preorder and postorder, display all traversal order.

```
#include<stdio.h>
#include<stdlib.h>

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

Node* createNode() {
    struct Node* node = (struct Node*) malloc
                        (sizeof (struct Node));
    int value;
    printf(" Enter value: ");
    scanf ("%d", &value);
    node -> data = value;
    node -> left = node -> right = NULL;
    return node;
}
```

Page No :

```
void insert (Node* root, Node* temp)
{
    if(temp -> data < root -> data){
        if (root -> left != NULL) {
            insert(root -> left, temp);
        }
        else {
            root -> left = temp;
        }
    }

    if (temp -> data > root -> data) {
        if (root -> right != NULL) {
            insert (root -> right, temp);
        }
        else {
            root -> right = temp;
        }
    }
}

void preorder (Node* root) {
    if (root != NULL) {
        printf ("%d", root -> data);
        preorder(root -> left);
        preorder (root -> right);
    }
}
```

```
void postorder (Node* root)
{
    if (root != NULL){
        postorder(root -> left);
        postorder (root -> right);
        printf("%d ", root -> data);
    }
}

void inorder (Node* root){
    if (root != NULL){
        inorder (root -> left);
        printf("%d ", root -> data);
        inorder( root -> right);
    }
}

int main(){
    char ch;
    Node* root = NULL;

    do{
        Node* temp = createnode();
        if (root == NULL)
            root = temp;
        else
            insert(root, temp);
        printf("Do you want to continue? If Yes then
                enter Y/y: ");
        scanf(" %c", &ch);
    }while(ch == 'Y' || ch == 'y');
    printf("Preorder traversal: ");
    preorder(root);
    printf("\n");
    printf("Inorder traversal: ");
    inorder(root);
    printf("\n");
    printf("Postorder traversal: ");
    postorder(root);
    printf("\n");
    return 0;
}
```

Output:

```
Enter Value: 7
Do you want to continue? If Yes then enter Y/y
Enter Value: 6
y
Enter Value: 3
y
Enter Value 4
y
Enter value: 1
n
Preorder Traversal : 7 6 3 1 4
Inorder Traversal : 1 3 4 6 7
Posforder Traversal : 1 4 3 6 7
```

Code:

```
#include<stdio.h>
#include<stdlib.h>

struct Node{
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(){
    struct Node* node=(struct Node*)malloc(sizeof(struct Node));
    int value;
    printf("Enter value:");
    scanf("%d",&value);
```

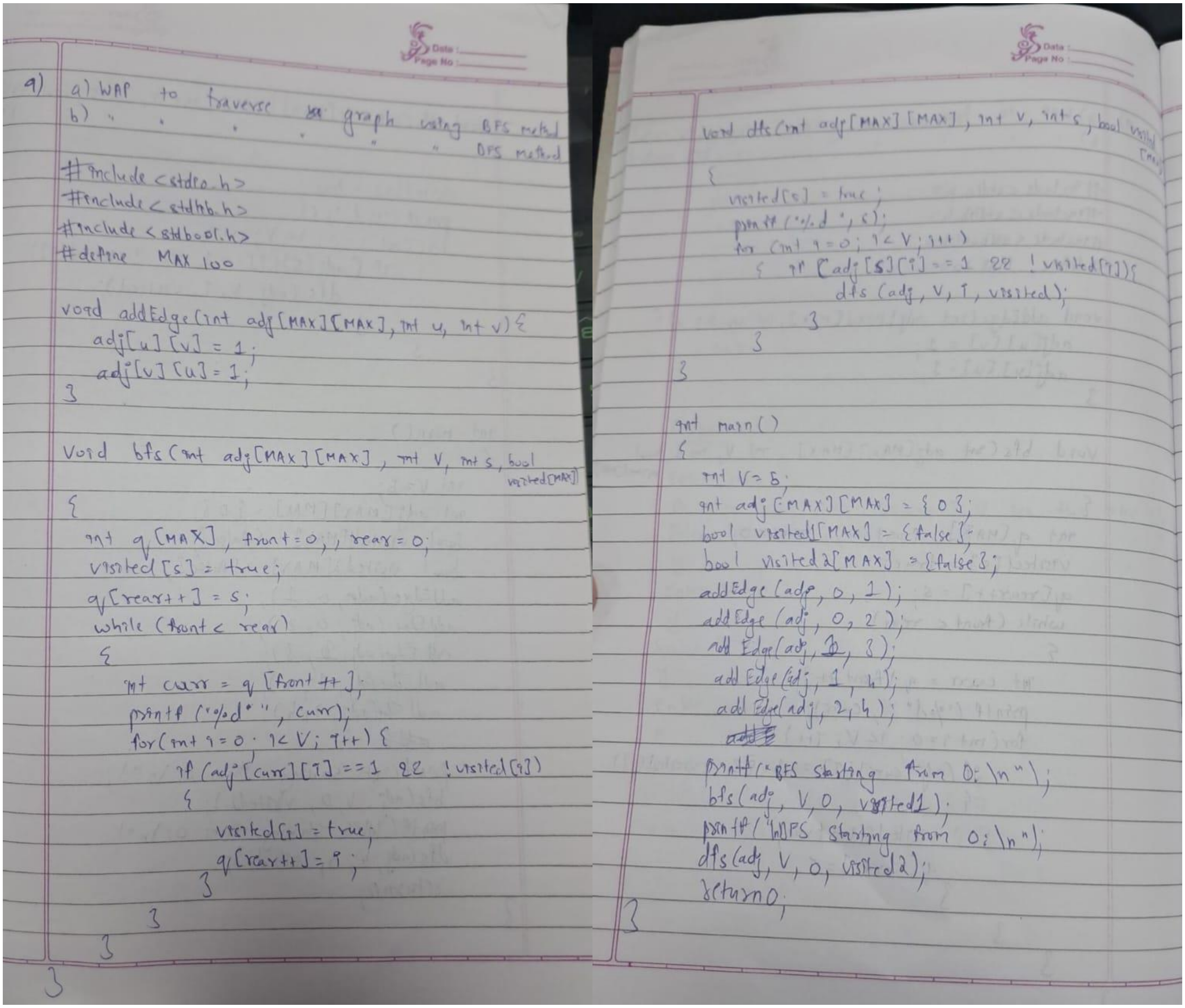
```
    node->data=value;
    node->left=node->right=NULL;
    return node;
}

void insert(struct Node* root,struct Node* temp){
    if(temp->data<root->data){
        if(root->left!=NULL){
            insert(root->left,temp);
        }
        else{
            root->left=temp;
        }
    }

    if(temp->data>root->data){
        if(root->right!=NULL){
            insert(root->right,temp);
        }
        else{
            root->right=temp;
        }
    }
}

void preorder(struct Node* root){
    if(root!=NULL){
    printf("%d ",root->data);
    preorder(root->left);
    preorder(root->right);
    }
}

void postorder(struct Node* root){
    if(root!=NULL){
    postorder(root->left);
    postorder(root->right);
    printf("%d ",root->data);
    }
}

void inorder(struct Node* root){
    if(root!=NULL){
    inorder(root->left);
    printf("%d ",root->data);
    inorder(root->right);
    }
}
```

```
int main(){
    char ch;
    struct Node* root=NULL;

    do{
        struct Node*temp=createNode();
        if(root==NULL){
            root=temp;
        }
        else{
            insert(root,temp);
        }
        printf("Do you want to continue? if Yes the enter Y/y:");
        scanf(" %c",&ch);
    }while(ch=='Y' || ch=='y');

    printf("Preorder traversal:");
    preorder(root);
    printf("\n");
    printf("Inorder traversal:");
    inorder(root);
    printf("\n");
    printf("Postorder traversal:");
    postorder(root);
    printf("\n");
    return 0;
}
```

```
Enter value:7
Do you want to continue? if Yes the enter Y/y:y
Enter value:6
Do you want to continue? if Yes the enter Y/y:y
Enter value:3
Do you want to continue? if Yes the enter Y/y:y
Enter value:4
Do you want to continue? if Yes the enter Y/y:y
Enter value:1
Do you want to continue? if Yes the enter Y/y:n
Preorder traversal:7 6 3 1 4
Inorder traversal:1 3 4 6 7
Postorder traversal:1 4 3 6 7
```

Output

9a) Write a program to traverse a graph using BFS method.
9b) Write a program to traverse a graph using DFS method.



9) a) WAP to traverse graph using BFS method
b) " " " " " DFS method

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#define MAX 100

void addEdge(int adj[MAX][MAX], int u, int v){
    adj[u][v] = 1;
    adj[v][u] = 1;
}

void bfs(int adj[MAX][MAX], int V, int s, bool visited[MAX])
{
  int q[MAX], front=0, rear=0;
  visited[s] = true;
  q[rear++] = s;
  while (front < rear)
  {
     int curr = q[front++];
     printf("%d ", curr);
     for(int i=0; i<V; i++) {
        if (adj[curr][i] ==1 && !visited[i])
        {
           visited[i] = true;
           q[rear++] = i;
        }
     }
  }
}

void dfs(int adj[MAX][MAX], int V, int s, bool visited[MAX])
{
   visited[s] = true;
   printf("%d ", s);
   for (int i=0; i<V; i++)
   {  if (adj[s][i] == 1 && !visited[i]){
         dfs(adj, V, i, visited);
      }
   }
}

int main()
{
   int V=5;
   int adj[MAX][MAX] = {0};
   bool visited1[MAX] = {false};
   bool visited2[MAX] = {false};
   addEdge(adj, 0, 1);
   addEdge(adj, 0, 2);
   addEdge(adj, 1, 3);
   addEdge(adj, 1, 4);
   addEdge(adj, 2, 4);
   printf("BFS Starting from 0: \n");
   bfs(adj, V, 0, visited1);
   printf("\nDFS Starting from 0: \n");
   dfs(adj, V, 0, visited2);
   return 0;
}
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#define MAX 100
```

```
void addEdge(int adj[MAX][MAX], int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1; // Undirected graph
}

void bfs(int adj[MAX][MAX], int V, int s,bool visited[MAX]) {
// Create a queue for BFS
    int q[MAX], front = 0, rear = 0;
// Mark the source node as visited and enqueue it
    visited[s] = true;
    q[rear++] = s;
    while (front < rear) { // Iterate over the queue
    // Dequeue a vertex and print it
    int curr = q[front++];
    printf("%d ", curr);

// Get all adjacent vertices of the dequeued vertex
// If an adjacent has not been visited, mark it visited and enqueue it
    for (int i = 0; i < V; i++) {
        if (adj[curr][i] == 1 && !visited[i]) {
            visited[i] = true;
            q[rear++] = i;
            }
        }
    }
}

// Function to perform DFS using recursion

void dfs(int adj[MAX][MAX], int V, int s,bool visited[MAX]) {
    // Mark the current node as visited
    visited[s] = true;
    // Print the current node
    printf("%d ", s);
    // Recur for all the vertices adjacent to the current node
    for (int i = 0; i < V; i++) {
    // If the vertex is adjacent and not visited, recurse on it
        if (adj[s][i] == 1 && !visited[i]) {
        dfs(adj, V, i, visited);
        }
    }
}

int main() {
    int V = 5;
// Adjacency matrix representation of the
    int adj[MAX][MAX] = {0};
```

```
    bool visited1[MAX] = {false};
    bool visited2[MAX] = {false};
// Add edges to the graph
    addEdge(adj, 0, 1);
    addEdge(adj, 0, 2);
    addEdge(adj, 1, 3);
    addEdge(adj, 1, 4);
    addEdge(adj, 2, 4);
    printf("BFS starting from 0:\n");
    bfs(adj, V, 0,visited1);
    printf("\nDFS starting from 0:\n");
    dfs(adj, V, 0,visited2);
    return 0;

}
```

```
BFS starting from 0:
0 1 2 3 4
DFS starting from 0:
0 1 3 4 2
```

Output